SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATION

A Crowdsourced, Client-Side AI Content Detection Extension for YouTube

Google Chrome Manifest V3 Desktop Extension • Main Input Collection & Extensible OOP Blueprint

Final Object-Oriented Implementation Plan for Active-Watch Evidence Collection

Modular Domain Architecture, Deterministic Segment Scheduling, Pluggable Mock Adapters for Unresolved Contracts, and Full-Scope Extension Hooks

This implementation blueprint translates the project abstract and authoritative revised scope into a production-ready Object-Oriented Programming (OOP) architecture for the Chrome Manifest V3 extension. It focuses on the **Main Input Collection Pipeline**—enforcing strict active-watch gating, deterministic timestamped segmentation, half-open observed interval attribution, bounded visible-tab screenshot and segment-scoped audio capture, deduplicated transcript extraction, and separated policy/browser permission state—while isolating every unresolved external dependency behind mockable adapter interfaces so the full product scope can be built and tested immediately.

Core Architectural Guarantees

Architectural Pillar	OOP Pattern & Primary Classes	Scope Requirement Enforced
Strict Active-Watch Gate	Composite Specification & Observer (`ActiveWatchGate`, `IGateSpecification`)	Zero background collection; halts within <50ms on pause, tab switch, hidden DOM, or policy/permission revocation.
Partial Interval Attribution	Immutable Value Objects & State Machine (`TimeInterval`, `ObservedIntervalTracker`)	Records only observed sub-intervals (e.g., `[10, 30)` & `[30, 48)`); never marks a whole 30s segment complete upon entry.
Mock Adapters for Unknowns	Adapter & Strategy Patterns (`IBackendApiAdapter`, `IAudioCaptureAdapter`, etc.)	Encapsulates all 10 unresolved Section 16 decisions behind deterministic mock and configurable production adapters.
Full-Scope & Platform Extensibility	Open/Closed Principle & Abstract Factory (`IPlatformAdapter`, `IFilterStrategy`)	Allows on-page badges, feed filtering, and future social platforms (TikTok, Reels, X) to plug in with zero core mutations.

Group 3 Team Member	Roll Number	Primary Architectural Ownership	Target Milestone
Lakshya Dubey	2024BCS0339	Core Domain Model, MV3 Orchestrator & Active-Watch Gate	Milestones 1, 3, 5
V Kannabiran	2024BCS0215	Screenshot & Tab Audio Capture Adapters + Feasibility Spike	Milestones 2, 5, 8
Armaan Z Hussain	2024BCS0323	Transcript Extractor, Backend API Adapter & Cache Coordination	Milestones 4, 6, 8
Nishanth M	2024BEC0023	Policy/Permission Managers, Badge Observers & Filter Strategies	Milestones 3, 7, 8

1. Executive Architectural Strategy: Separating Lightweight Collection from Heavy Inference

Centralized scanning of YouTube at platform scale is computationally prohibitive and bandwidth-intensive. As established in the project abstract and the revised authoritative scope, our system crowdsources evidence collection organically: the first active viewer of an unmapped or partially mapped video contributes bounded client-side evidence for the specific timestamp intervals they actually watch, while a backend structuring workflow and AI inference service compute and cache the resulting **AI-reliance score** for all subsequent viewers.

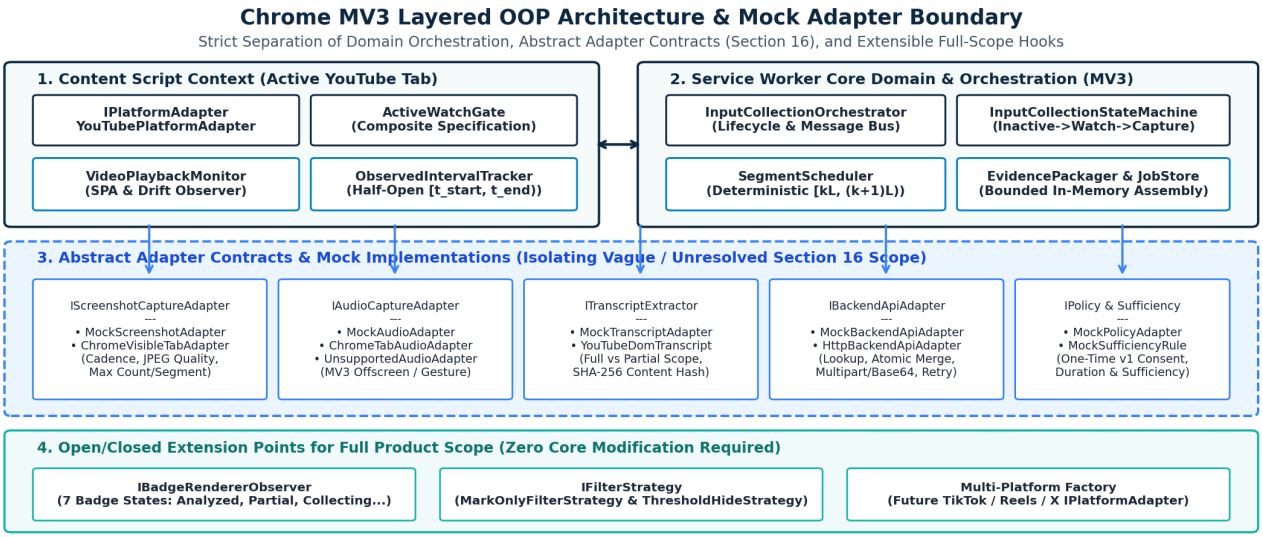


Figure 1: Four-layer Object-Oriented architecture across Chrome Manifest V3 execution contexts, illustrating how abstract adapter contracts decouple core input collection logic from unresolved Section 16 parameters and downstream UI/platform extensions.

To guarantee that the client extension remains lightweight, non-disruptive to playback, and resilient to Chrome Manifest V3 (MV3) service worker lifecycle termination, the extension architecture is organized around five Object-Oriented design principles:

- **Composite Specification & Observer Pattern for the Active-Watch Gate:** Every condition required for legal and valid capture (extension enabled, policy version matched, contribution toggle active, tab focused, document visible, video playing with monotonic clock advancement, missing segments reported by backend, and Chrome capture permission granted) is modeled as an independent `IGateSpecification` object observed by `ActiveWatchGate`. Any single specification failure triggers an immediate `GateClosedEvent`, halting capture and flushing only completed bounded intervals.
- **Adapter Pattern for Unresolved Contracts & Browser Constraints:** Section 16 of the specification identifies ten critical parameters and contracts that must not be hardcoded (backend schemas, upload encoding, segment duration, sufficiency rules, screenshot cadence/dimensions, MV3 tab-audio feasibility, transcript extraction selectors, policy version propagation, partial score semantics, and cache TTL). Each unknown is isolated behind a strict TypeScript interface paired with a **Mock Adapter** (for immediate end-to-end development and automated testing) and a **Production Adapter** driven by runtime configuration.
- **Immutable Value Objects for Interval Math:** Half-open playback intervals `[startSeconds, endSeconds)` are encapsulated in an immutable `TimeInterval` class that enforces boundary clamping, overlap detection, gap subtraction, and contiguous interval merging—preventing off-by-one errors or false full-segment coverage claims.
- **State Pattern for Contribution Lifecycle:** Collection transitions deterministically across six concrete state classes (`InactiveState`, `CheckingCacheState`, `ActiveWatchingState`, `CapturingState`, `UploadingState`, and `CompletedState`), persisting minimal metadata in `chrome.storage.local` so ephemeral MV3 service workers resume cleanly without storing raw media on disk.
- **Open/Closed Extensibility for Full Product Scope & Future Platforms:** Platform-specific DOM logic is encapsulated inside `IPlatformAdapter` (implemented in V1 by `YouTubePlatformAdapter`), while downstream presentation features (the 7 badge states and opt-in threshold filtering) subscribe to the core collection pipeline via `ICollectionEventListener` and `IFilterStrategy` without modifying input collection classes.

2. Chrome Manifest V3 Context Topology & Package Structure

Because Chrome Manifest V3 strictly separates DOM access (Content Scripts), privileged tab/window APIs (Service Worker), and media stream recording (Offscreen Document / Tab Context), our OOP class hierarchy maps cleanly onto isolated runtime contexts communicating over a strongly typed `IMessageBus` contract. No class in the domain layer depends directly on global `chrome.*` namespaces; all browser primitives are injected via interfaces.

MV3 Execution Context	Core OOP Classes Hosted	Primary Responsibilities & MV3 Constraints Handled
Content Script (`youtube.com/*`)	`YouTubePlatformAdapter`, `VideoPlaybackMonitor`, `ActiveWatchGate`, `ObservedIntervalTracker`, `YouTubeDomTranscriptAdapter`, `BadgeAndFilterController`	Observes YouTube SPA navigation (`yt-navigate-finish`), binds to the active `HTMLVideoElement`, tracks monotonic `currentTime` advancement, evaluates DOM/tab visibility specifications, extracts accessible transcripts, and renders on-page badges/filters. Never holds API secrets.
Service Worker (`background.ts`)	`InputCollectionOrchestrator`, `InputCollectionStateMachine`, `SegmentScheduler`, `ChromeVisibleTabScreenshotAdapter`, `EvidencePackager`, `ResumableJobStateStore`, `HttpBackendApiAdapter`	Stateless-resilient coordinator. Performs deduplicated backend cache lookups, schedules missing segment capture, invokes `chrome.tabs.captureVisibleTab` when gated, assembles bounded `EvidencePackage` payloads in memory, manages exponential backoff uploads (honoring `Retry-After`), and persists lightweight metadata to `chrome.storage.local`.
Offscreen / Audio Context (`offscreen.html`)	`ChromeTabAudioCaptureAdapter`, `AudioTimelineAligner`, `UnsupportedAudioCaptureAdapter`	Receives a stream ID from `chrome.tabCapture.getMediaStreamId` after an explicit user gesture, routes audio to destination so playback remains audible to the user, slices bounded `audio/webm` chunks aligned to video timestamps, and immediately aborts stream tracks on `GateClosedEvent`.
Popup & Onboarding UI (`popup.html`, `onboarding.html`)	`PolicyConsentManager`, `BrowserPermissionCoordinator`, `UserPreferencesRepository`	Presents one-time versioned collection policy acceptance (stored by `accepted_policy_version`), provides the explicit user-gesture *"Enable Contribution"* button for Chrome tab capture authorization, and exposes filtering mode (`mark_only` vs. opt-in hide threshold) and live status.

Recommended Repository Directory Layout

```
src/
├── domain/
│   ├── models/              # VideoIdentity, TimeInterval, VideoSegment, EvidencePackage, PolicyState
│   ├── specifications/      # IGateSpecification, ActiveWatchGate, 9 concrete gate specifications
│   ├── state/               # ICollectionState, InputCollectionStateMachine, Concrete States
│   └── services/            # SegmentScheduler, ObservedIntervalTracker, EvidencePackager
├── ports/                   # Abstract OOP Interfaces (Section 16 & Extensibility Contracts)
│   ├── IPlatformAdapter.ts
│   ├── IBackendApiAdapter.ts
│   ├── IScreenshotCaptureAdapter.ts
│   ├── IAudioCaptureAdapter.ts
│   ├── ITranscriptExtractorAdapter.ts
│   ├── ISegmentationStrategy.ts
│   ├── IPolicyAndPermissionAdapter.ts
│   └── IBadgeAndFilterExtension.ts
├── adapters/
│   ├── mocks/               # Deterministic Mock Adapters for all vague/unresolved scope items
│   ├── youtube/             # YouTubePlatformAdapter, YouTubeDomTranscriptAdapter
│   ├── chrome/              # ChromeVisibleTabScreenshotAdapter, ChromeTabAudioCaptureAdapter, StorageRepo
│   └── http/                # HttpBackendApiAdapter (JSON Base64 & Multipart strategies)
├── extensions/              # Full-scope modules built atop core events (Badges, Filter, Popup)
└── contexts/                # MV3 entrypoints: content.ts, background.ts, offscreen.ts, popup.ts
```

3. Core Domain Model & Immutable Value Objects

The domain layer enforces every invariant defined in Sections 1, 3, and 6 of `extension_scope.md` using pure TypeScript classes with zero browser or network dependencies. By making `TimeInterval` and `EvidencePackage` immutable value objects, the extension mathematically prevents partial observations from being misreported as full-segment coverage.

3.1 Half-Open Interval Arithmetic (`TimeInterval` & `VideoSegment`)

All video segments and observed playback slices use half-open interval notation `[startSeconds, endSeconds)`. If a video is 120 seconds long with 30-second segments and Viewer A watches from `10.0s` to `48.0s` before switching tabs, `ObservedIntervalTracker` splits the continuous watch session at the `30.0s` segment boundary into two exact intervals: `TimeInterval(10.0, 30.0)` for Segment 0 and `TimeInterval(30.0, 48.0)` for Segment 1.

```
export class TimeInterval {
  public readonly startSeconds: number;
  public readonly endSeconds: number;

  constructor(startSeconds: number, endSeconds: number) {
    if (!Number.isFinite(startSeconds) || !Number.isFinite(endSeconds) || startSeconds < 0) {
      throw new RangeError(`Invalid interval bounds: [${startSeconds}, ${endSeconds})`);
    }
    if (endSeconds <= startSeconds) {
      throw new RangeError(`Half-open interval requires endSeconds > startSeconds: [${startSeconds},
${endSeconds})`);
    }
    this.startSeconds = Number(startSeconds.toFixed(3));
    this.endSeconds = Number(endSeconds.toFixed(3));
  }

  public get durationSeconds(): number {
    return Number((this.endSeconds - this.startSeconds).toFixed(3));
  }

  public contains(timestampSeconds: number): boolean {
    return timestampSeconds >= this.startSeconds && timestampSeconds < this.endSeconds;
  }

  public intersect(other: TimeInterval): TimeInterval | null {
    const start = Math.max(this.startSeconds, other.startSeconds);
    const end = Math.min(this.endSeconds, other.endSeconds);
    return end > start ? new TimeInterval(start, end) : null;
  }

  public static mergeContiguous(intervals: ReadonlyArray<TimeInterval>, epsilonSeconds = 0.25): TimeInterval[] {
    if (intervals.length === 0) return [];
    const sorted = [...intervals].sort((a, b) => a.startSeconds - b.startSeconds);
    const merged: TimeInterval[] = [sorted[0]];
    for (let i = 1; i < sorted.length; i++) {
      const prev = merged[merged.length - 1];
      const curr = sorted[i];
      if (curr.startSeconds <= prev.endSeconds + epsilonSeconds) {
        merged[merged.length - 1] = new TimeInterval(prev.startSeconds, Math.max(prev.endSeconds,
curr.endSeconds));
      } else {
        merged.push(curr);
      }
    }
    return merged;
  }
}

export type CoverageStatus = 'missing' | 'partial' | 'complete';

export class VideoSegment {
  constructor(
    public readonly videoId: string,
    public readonly index: number,
    public readonly bounds: TimeInterval,
    public readonly backendCoverage: CoverageStatus,
    public readonly observedIntervals: ReadonlyArray<TimeInterval> = []
  ) {}
```

```
  public clampObservation(rawInterval: TimeInterval): TimeInterval | null {
    return this.bounds.intersect(rawInterval);
  }

  public withAddedObservation(rawInterval: TimeInterval): VideoSegment {
    const clamped = this.clampObservation(rawInterval);
    if (!clamped) return this;
    const merged = TimeInterval.mergeContiguous([...this.observedIntervals, clamped]);
    return new VideoSegment(this.videoId, this.index, this.bounds, this.backendCoverage, merged);
  }
}
```

3.2 Multimodal Evidence Entities & Privacy-Safe Packaging

To satisfy Requirement 8 and Section 12 (no raw media persisted in `chrome.storage.local`, no fabrication of uncaptured modalities, and explicit capability reporting), `EvidencePackage` is constructed strictly in volatile memory and discarded immediately after upload completion or non-recoverable gate closure.

```
export type TranscriptScope = 'full' | 'partial' | 'unavailable';

export interface ScreenshotSample {
  readonly videoTimestampSeconds: number;
  readonly mimeType: 'image/jpeg' | 'image/webp';
  readonly data: Uint8Array | string; // Binary buffer for Multipart or Base64 string
  readonly width: number;
  readonly height: number;
}

export interface AudioChunkSample {
  readonly interval: TimeInterval;
  readonly mimeType: string;          // e.g., 'audio/webm;codecs=opus'
  readonly data: Uint8Array | string;
  readonly alignmentDriftMs: number;
}

export interface TranscriptSegmentDto {
  readonly startSeconds: number;
  readonly endSeconds: number;
  readonly text: string;
}

export interface TranscriptPayload {
  readonly scope: TranscriptScope;
  readonly language: string | null;
  readonly segments: ReadonlyArray<TranscriptSegmentDto>;
  readonly contentHash: string | null; // SHA-256 hash for backend deduplication
  readonly omissionReason?: string;
}

export interface CaptureCapabilities {
  readonly screenshots: boolean;
  readonly tabAudio: boolean;
  readonly transcriptScope: TranscriptScope;
  readonly audioUnavailableReason?: 'permission_denied' | 'gesture_required' | 'unsupported_browser' | null;
}

export class EvidencePackage {
  constructor(
    public readonly videoId: string,
    public readonly policyVersion: string,
    public readonly segmentIndex: number,
    public readonly segmentBounds: TimeInterval,
    public readonly observedIntervals: ReadonlyArray<TimeInterval>,
    public readonly frames: ReadonlyArray<ScreenshotSample>,
    public readonly audioChunks: ReadonlyArray<AudioChunkSample>,
    public readonly transcript: TranscriptPayload | null,
    public readonly capabilities: CaptureCapabilities
  ) {
    if (this.observedIntervals.length === 0) {
      throw new Error('Cannot build EvidencePackage without at least one verified observed interval.');
    }
```

```
  }
}
```

---

4. Main Input Collection Pipeline: Core OOP Classes & State Machine

The main input collection pipeline coordinates four core domain classes: `ActiveWatchGate`, `SegmentScheduler`, `ObservedIntervalTracker`, and `InputCollectionStateMachine`. Together, they guarantee that capture occurs exclusively during verified active watching of uncovered video segments and halts instantaneously when any condition changes.

4.1 The Composite `ActiveWatchGate` Specification

Section 5 of `extension_scope.md` mandates nine distinct preconditions that must hold simultaneously before a single screenshot or audio byte may be captured. Rather than scattering `if` statements across content scripts and service workers, we implement the **Composite Specification Pattern** combined with an event emitter:

```
export interface WatchContextSnapshot {
  readonly extensionEnabled: boolean;
  readonly acceptedPolicyVersion: string | null;
  readonly currentPolicyVersion: string;
  readonly contributionEnabled: boolean;
  readonly isTabActiveInWindow: boolean;
  readonly documentVisibility: DocumentVisibilityState;
  readonly isVideoPlaying: boolean;
  readonly isPlaybackTimeAdvancing: boolean; // Verified via monotonic delta > 0 across 250ms ticks
  readonly isVideoEndedOrSeeking: boolean;
  readonly hasMissingBackendSegments: boolean;
  readonly hasScreenshotOrAudioCapability: boolean;
}

export interface IGateSpecification {
  readonly id: string;
  isSatisfiedBy(ctx: WatchContextSnapshot): boolean;
  failureReason(ctx: WatchContextSnapshot): string | null;
}

export class ActiveWatchGate {
  private readonly specs: ReadonlyArray<IGateSpecification>;
  private readonly listeners = new Set<(passed: boolean, failedReasons: string[]) => void>();
  private lastPassedState = false;

  constructor(specs: ReadonlyArray<IGateSpecification>) {
    this.specs = specs;
  }

  public evaluate(ctx: WatchContextSnapshot): { passed: boolean; failedReasons: string[] } {
    const failedReasons: string[] = [];
    for (const spec of this.specs) {
      if (!spec.isSatisfiedBy(ctx)) {
        failedReasons.push(spec.failureReason(ctx) ?? spec.id);
      }
    }
    const passed = failedReasons.length === 0;
    if (passed !== this.lastPassedState || !passed) {
      this.lastPassedState = passed;
      this.listeners.forEach(listener => listener(passed, failedReasons));
    }
    return { passed, failedReasons };
  }

  public onGateChange(listener: (passed: boolean, failedReasons: string[]) => void): () => void {
    this.listeners.add(listener);
    return () => this.listeners.delete(listener);
  }
}
```

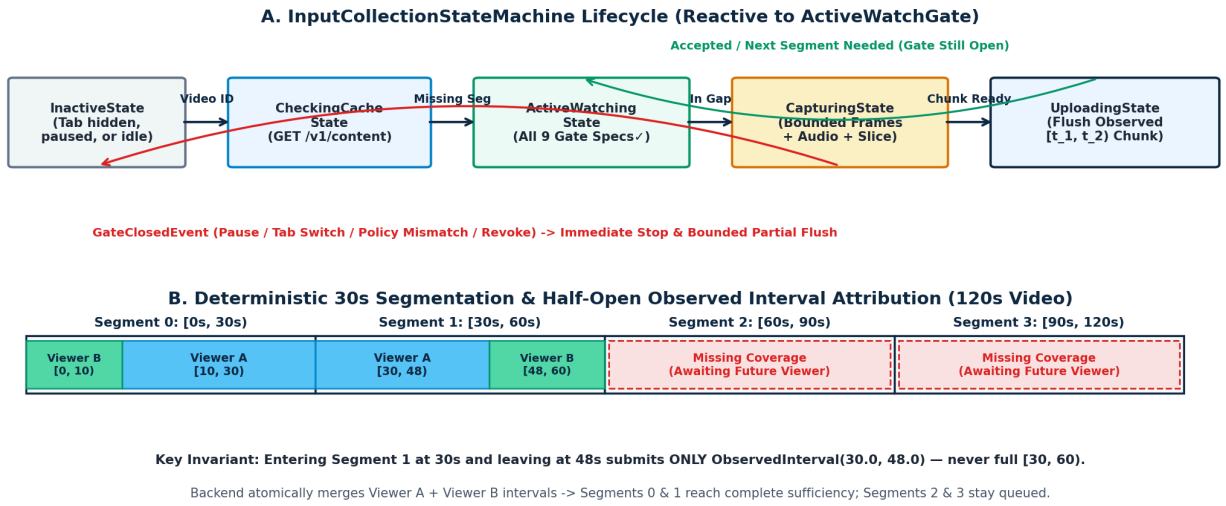


Figure 2: (A) Reactive state transitions inside InputCollectionStateMachine governed by ActiveWatchGate specifications; (B) Partial interval attribution across 30-second segments showing how Viewer A and Viewer B contributions merge without over-claiming coverage.

Concrete Specification Class	Observable Browser / State Signal	Immediate Stop Trigger & Flush Behavior
`PolicyVersionMatchSpec`	`acceptedPolicyVersion === currentPolicyVersion`	Fires on policy update or withdrawal; aborts capture and **discards** unuploaded chunks.
`UserContributionEnabledSpec`	`enabled && contribution_enabled` in storage	Fires on popup toggle off; halts capture immediately and discards in-flight buffer.
`TabAndDocumentVisibleSpec`	`chrome.tabs.onActivated`, `chrome.windows.onFocusChanged`, `document.visibilityState === 'visible'`	Fires within <50ms of tab switch or window minimize; closes current `TimeInterval` and flushes valid completed chunk.
`VideoAdvancingPlaybackSpec`	HTML5 `playing`, `pause`, `ended`, `seeking` + 250ms monotonic `currentTime` check	Detects pause, buffer stall, or user seek; closes current observed interval at exact pre-seek/pause timestamp and flushes.
`MissingSegmentTargetSpec`	`SegmentScheduler.isCurrentTimestampInMissingSegment(t)`	Transitions from `CapturingState` to `ActiveWatchingState` (idle watch) when playback enters an already-complete segment.
`CaptureCapabilityAuthorizedSpec`	`screenshotPermitted || audioPermitted`	Fires on permission revocation or stream error; degrades gracefully without disrupting YouTube playback.

4.2 Deterministic Segment Scheduling & Playback Drift Guard (`SegmentScheduler`)

`SegmentScheduler` uses an injected `ISegmentationStrategy` (defaulting to 30-second half-open segments `[0, 30)`, `[30, 60)`, …, `[k*L, min((k+1)*L, duration))`) synchronized with the backend response from `GET /v1/content/youtube/{video_id}`. During active playback, `ObservedIntervalTracker` samples `video.currentTime` every 250ms:

- **Normal Playback Advancement:** If `0 < deltaVideoTime <= deltaWallClock * playbackRate + 0.35s`, the current open observation interval `[t_start, t_current)` is extended.
- **User Seek or Jump Detection:** If the viewer scrubs the timeline (e.g., jumps from `18.0s` to `42.0s`), the tracker immediately seals `TimeInterval(t_start, 18.0)` in Segment 0, triggers a bounded partial flush for Segment 0, and opens a fresh interval `TimeInterval(42.0, ...)` inside Segment 1—guaranteeing that skipped timestamps `[18.0, 42.0)` are never claimed as watched.
- **Boundary Rollover:** When playback naturally crosses a segment boundary (e.g., `29.9s -> 30.1s`), the tracker seals `TimeInterval(t_start, 30.0)` for Segment 0, dispatches Segment 0's bounded `EvidencePackage` to the Service Worker upload queue, and initializes Segment 1 tracking at `30.0s` if Segment 1 is also listed in `missing_segment_indices`.

---

5. Abstract Interfaces & Mock Adapters for Vague / Unresolved Scope (Section 16)

Section 16 of `extension_scope.md` explicitly lists ten decisions that the implementation worker must confirm with the backend owner or validate via a technical spike rather than hardcoding. Our OOP design resolves this architectural tension by defining **six port interfaces** in `src/ports/` and providing **deterministic Mock Adapters** alongside configurable **Production Adapters**. Every unknown parameter is injected via a typed configuration object so swapping from mock to production requires zero changes to domain orchestration code.

Unresolved Scope Item (Section 16)	OOP Port Interface	Mock Adapter (Dev / Test / Spike)	Production Adapter (Configurable)
1. Backend API URL, Auth, Rate Limits & Schemas	`IBackendApiAdapter` `IPayloadSerializer`	`MockBackendApiAdapter` (in-memory DB, merges intervals, simulates `429 Retry-After`)	`HttpBackendApiAdapter` (supports Bearer/Signed-Token auth, Multipart or JSON Base64)
2. Segment Duration & Sufficiency Rule	`ISegmentationStrategy` `ISufficiencyPolicyAdapter`	`MockSufficiencyPolicyAdapter` (configurable e.g. ≥80% interval + ≥2 frames)	`BackendDrivenSegmentationStrategy` (reads `segment_duration_seconds` from lookup)
3. Coverage Tracking Granularity	`ICoverageTracker`	`HybridSegmentIntervalTracker` (fixed segments for scheduling + exact `TimeInterval[]`)	`HybridSegmentIntervalTracker` (serializes both `segment_index` and `observed_intervals`)
4. Screenshot Cadence, Dimensions & Max Count	`IScreenshotCaptureAdapter`	`MockScreenshotCaptureAdapter` (generates synthetic JPEG bytes with timestamp metadata)	`ChromeVisibleTabScreenshotAdapter` (`captureVisibleTab` + `OffscreenCanvas` resize)
5. Audio Format, Bitrate, Drift & MV3 UX	`IAudioCaptureAdapter`	`MockAudioCaptureAdapter` & `UnsupportedAudioCaptureAdapter`	`ChromeTabAudioCaptureAdapter` (MV3 Offscreen `tabCapture` + loop-back audio routing)
6. Transcript Access & Full vs. Partial Scope	`ITranscriptExtractorAdapter`	`MockTranscriptExtractorAdapter` (configurable `full`, `partial`, or `unavailable` fixtures)	`YouTubeDomTranscriptAdapter` (reads open transcript DOM / caption track + SHA-256 hash)

Unresolved Scope Item (Section 16)	OOP Port Interface	Mock Adapter (Dev / Test / Spike)	Production Adapter (Configurable)
7. Policy Acceptance Scope & Version Delivery	`IPolicyConsentAdapter`	`MockPolicyConsentAdapter` (allows instant simulation of `v1 -> v2` version bump)	`StoragePolicyConsentAdapter` (syncs `current_policy_version` from backend header/config)
8. Partial Score Semantics & Cache TTL	`IResultCacheRepository`	`InMemoryResultCacheRepository` (configurable TTL and partial score visibility flag)	`ChromeStorageResultCacheRepository` (bounded LRU in `chrome.storage.local` with TTL)

5.1 Backend API Port & Stateful Mock Adapter (`IBackendApiAdapter`)

Because the final endpoint URLs, authentication headers, and binary vs. Base64 upload encoding are pending backend team sign-off, `IBackendApiAdapter` decouples the orchestrator from HTTP transport details. Notably, `MockBackendApiAdapter` implements real interval-merging math in memory so two simulated viewers (Viewer A watching `[10, 48)` and Viewer B watching `[0, 10)` and `[48, 60)`) can be tested end-to-end without a live server:

```
export interface VideoLookupResponseDto {
  readonly videoId: string;
  readonly status: 'not_found' | 'partial' | 'processing' | 'complete' | 'insufficient_evidence';
  readonly label: 'AI_GENERATED' | 'HUMAN_MADE' | 'MIXED' | 'UNKNOWN';
  readonly aiRelianceScore: number | null;
  readonly confidence: number | null;
  readonly segmentDurationSeconds: number;
  readonly segments: ReadonlyArray<{
    index: number;
    startSeconds: number;
    endSeconds: number;
    coverage: CoverageStatus;
  }>;
  readonly missingSegmentIndices: ReadonlyArray<number>;
  readonly analyzedAt: string | null;
  readonly requiredPolicyVersion?: string;
}

export interface ContributionUploadResponseDto {
  readonly contributionId: string;
  readonly acceptedIntervals: ReadonlyArray<{ startSeconds: number; endSeconds: number }>;
  readonly remainingIntervals: ReadonlyArray<{ startSeconds: number; endSeconds: number }>;
  readonly videoStatus: 'partial' | 'processing' | 'complete';
}

export interface IBackendApiAdapter {
  lookupVideoCoverage(videoId: string, signal?: AbortSignal): Promise<VideoLookupResponseDto>;
  submitContribution(pkg: EvidencePackage, signal?: AbortSignal): Promise<ContributionUploadResponseDto>;
}

export class MockBackendApiAdapter implements IBackendApiAdapter {
  private readonly videoStore = new Map<string, {
    durationSeconds: number;
    segmentDuration: number;
    observedBySegment: Map<number, TimeInterval[]>;
    transcriptHashes: Set<string>;
  }>();

  constructor(
    private readonly config = {
      defaultSegmentDurationSeconds: 30,
      defaultVideoDurationSeconds: 120,
      sufficiencyRatioThreshold: 0.95,
      simulateNetworkLatencyMs: 120,
      simulateRateLimitEveryNRequests: 0
    }
  ) {}
```

```
  public async lookupVideoCoverage(videoId: string): Promise<VideoLookupResponseDto> {
    await new Promise(r => setTimeout(r, this.config.simulateNetworkLatencyMs));
    const state = this.getOrCreateVideoState(videoId);
    const totalSegments = Math.ceil(state.durationSeconds / state.segmentDuration);
    const segments = [];
    const missingSegmentIndices: number[] = [];

    for (let i = 0; i < totalSegments; i++) {
      const start = i * state.segmentDuration;
      const end = Math.min((i + 1) * state.segmentDuration, state.durationSeconds);
      const segBounds = new TimeInterval(start, end);
      const observed = TimeInterval.mergeContiguous(state.observedBySegment.get(i) ?? []);
      const coveredSec = observed.reduce((acc, iv) => acc + iv.durationSeconds, 0);
      const ratio = coveredSec / segBounds.durationSeconds;

      const coverage: CoverageStatus =
        ratio >= this.config.sufficiencyRatioThreshold ? 'complete' : coveredSec > 0 ? 'partial' : 'missing';
      if (coverage !== 'complete') missingSegmentIndices.push(i);
      segments.push({ index: i, startSeconds: start, endSeconds: end, coverage });
    }

    const allComplete = missingSegmentIndices.length === 0;
    return {
      videoId,
      status: allComplete ? 'complete' : segments.some(s => s.coverage !== 'missing') ? 'partial' : 'not_found',
      label: allComplete ? 'AI_GENERATED' : 'UNKNOWN',
      aiRelianceScore: allComplete ? 0.86 : null,
      confidence: allComplete ? 0.91 : null,
      segmentDurationSeconds: state.segmentDuration,
      segments,
      missingSegmentIndices,
      analyzedAt: allComplete ? new Date().toISOString() : null
    };
  }

  public async submitContribution(pkg: EvidencePackage): Promise<ContributionUploadResponseDto> {
    await new Promise(r => setTimeout(r, this.config.simulateNetworkLatencyMs));
    const state = this.getOrCreateVideoState(pkg.videoId);
    if (pkg.transcript?.contentHash) {
      state.transcriptHashes.add(pkg.transcript.contentHash); // Deduplicated by hash
    }
    const existing = state.observedBySegment.get(pkg.segmentIndex) ?? [];
    const merged = TimeInterval.mergeContiguous([...existing, ...pkg.observedIntervals]);
    state.observedBySegment.set(pkg.segmentIndex, merged);

    const lookup = await this.lookupVideoCoverage(pkg.videoId);
    return {
      contributionId: `mock_contrib_${Date.now()}`,
      acceptedIntervals: merged.map(i => ({ startSeconds: i.startSeconds, endSeconds: i.endSeconds })),
      remainingIntervals: [], // Computed by subtracting merged intervals from segmentBounds
      videoStatus: lookup.status === 'complete' ? 'complete' : 'partial'
    };
  }

  private getOrCreateVideoState(videoId: string) {
    if (!this.videoStore.has(videoId)) {
      this.videoStore.set(videoId, {
        durationSeconds: this.config.defaultVideoDurationSeconds,
        segmentDuration: this.config.defaultSegmentDurationSeconds,
        observedBySegment: new Map(),
        transcriptHashes: new Set()
      });
    }
    return this.videoStore.get(videoId)!;
  }
}
```

5.2 Visible-Tab Screenshot & Segment-Scoped Audio Capture Adapters

Section 7 requires two distinct media capture pipelines that operate strictly inside the active-watch gate and respect Chrome's security model:

- **`IScreenshotCaptureAdapter`:** Configured via `ScreenshotCaptureConfig` (defaulting to 1 sample every 5 seconds of watched video time, maximum 6 frames per 30-second segment, downscaled to max width `1280px` at `0.75` JPEG quality via `OffscreenCanvas`, with a strict `150 KB` per-frame ceiling). `MockScreenshotCaptureAdapter` emits deterministic, small synthetic JPEG byte arrays tagged with exact `video.currentTime` timestamps for unit testing cadence and memory bounds.
- **`IAudioCaptureAdapter` & Explicit Fallback State:** In Chrome MV3, `chrome.tabCapture` requires an explicit user gesture and routes tab audio into a `MediaStream` (which mutes the tab unless explicitly connected back to an `AudioContext.destination` inside an Offscreen Document). To honor Section 7's mandate (*"If supported tab audio capture cannot be made reliable under the required UX, do not silently substitute full-video download... implement an explicit unsupported/unavailable state"*), our design provides three interchangeable implementations of `IAudioCaptureAdapter`:
  1. `ChromeTabAudioCaptureAdapter`: Captures bounded 5-second `audio/webm;codecs=opus` chunks at `32 kbps` in an MV3 offscreen document while looping the source node directly to `audioCtx.destination` so user playback remains completely audible, recording drift against `video.currentTime`.
  2. `MockAudioCaptureAdapter`: Generates synthetic timestamped audio chunks and supports simulated drift/permission revocation for automated test suites.
  3. `UnsupportedAudioCaptureAdapter`: A Null-Object / Graceful-Degradation adapter that immediately reports `capabilities.tabAudio = false` and `audioUnavailableReason = 'gesture_required' | 'unsupported_browser'`, allowing visual screenshot and transcript contributions to proceed seamlessly without breaking playback.

```
export interface IScreenshotCaptureAdapter {
  captureFrame(videoTimestampSeconds: number): Promise<ScreenshotSample | null>;
  abortPendingCapture(): void;
}

export interface IAudioCaptureAdapter {
  readonly isSupportedAndAuthorized: boolean;
  readonly unavailableReason: 'permission_denied' | 'gesture_required' | 'unsupported_browser' | null;
  startSegmentRecording(segmentBounds: TimeInterval, startVideoTime: number): Promise<void>;
  stopAndFlushRecording(endVideoTime: number): Promise<ReadonlyArray<AudioChunkSample>>;
  abortAndDiscard(): Promise<void>;
}
```

5.3 Transcript Extraction & Content-Hash Deduplication (`ITranscriptExtractorAdapter`)

Section 8 specifies that full-video transcripts are supplementary and must never be obtained by opening hidden players or bypassing access controls. `ITranscriptExtractorAdapter` defines a clean extraction contract with three distinct scope outcomes:

- **`scope: "full"`:** Extracted when YouTube's accessible transcript panel or exposed initial player caption track provides the complete timestamped cue list within configured size limits (default max `250 KB` or `2,500` cue segments). The adapter computes a Web Crypto `SHA-256` digest (`contentHash`) and caches the hash per `videoId` in memory so the full transcript is uploaded at most once per video session.
- **`scope: "partial"`:** Reported when no full transcript panel is accessible, but the user has visible closed captions active during playback; only the caption cues observed during the watched `TimeInterval` are packaged.
- **`scope: "unavailable"`:** Reported when the video has no captions or the transcript exceeds maximum size bounds without a backend chunking scheme (attaching an explicit `omissionReason`).

---

6. One-Time Policy Acceptance, Browser Permissions & MV3 Resumability

Requirements 6, 7, 8, and Section 9 establish a strict separation between **Policy Acceptance** (legal/product consent to collect and upload crowdsourced evidence) and **Browser Permissions** (Chrome technical authorization for `activeTab` / `tabCapture`).

Concern	Responsible OOP Class	Implementation Rules & Lifecycle Behavior
One-Time Policy Acceptance	`PolicyConsentManager`	Stores `accepted_policy_version` (e.g., `"v1"`) and ISO-8601 `accepted_at` in `chrome.storage.local` during onboarding. Never prompts per video. If backend/extension updates `currentPolicyVersion` to `"v2"`, `PolicyVersionMatchSpec` immediately closes `ActiveWatchGate` until the user accepts `"v2"` in the popup/onboarding banner.

Concern	Responsible OOP Class	Implementation Rules & Lifecycle Behavior
Browser Capability & User Gesture	`BrowserPermissionCoordinator`	Separately tracks runtime Chrome permissions and active-tab user gestures. Provides a dedicated *"Enable Contribution for This Session"* button in the extension popup so user-gesture requirements for tab audio capture are satisfied transparently without claiming policy consent grants browser permissions.
Ephemeral MV3 State & Privacy	`ResumableJobStateStore`	Persists only lightweight metadata (`videoId`, `segmentIndex`, already-accepted `TimeInterval[]`, and `transcriptHashSent`) in `chrome.storage.local`. Raw screenshots, audio buffers, and transcript text are **never** written to storage or `console.log`.

7. Extensibility Blueprint: Building the Full Scope & Future Platforms

A core requirement of this implementation plan is that the entire product scope—and future expansion to other social media platforms—can be built cleanly on top of the input collection design without refactoring core classes. We achieve this through three dedicated extension interfaces:

7.1 On-Page Badge Presentation (`IBadgeRendererObserver`)

Section 11 defines seven mandatory badge states: `Analyzed`, `Partially analyzed`, `Not analyzed`, `Collecting evidence`, `Processing`, `Insufficient evidence`, and `Unavailable`. Rather than coupling badge DOM rendering to the state machine, `BadgePresenter` implements `ICollectionStateObserver` and maps domain events deterministically to UI badges—ensuring partial evidence is never displayed as a definitive full-video score.

7.2 Strategy-Based Content Filtering (`IFilterStrategy`)

Filtering on YouTube watch pages and feed cards uses the **Strategy Pattern**:

- **`MarkOnlyFilterStrategy` (Default):** Renders non-intrusive status badges on video cards and players without hiding any content.
- **`ThresholdHideFilterStrategy` (Opt-In):** Applies a reversible CSS overlay/collapse only when `lookup.status === 'complete' && lookup.aiRelianceScore !== null && lookup.aiRelianceScore >= userThreshold`. Explicitly ignores `partial`, `UNKNOWN`, pending, or failed lookups (fail-open invariant) and exposes a one-click `revealForSession(videoId)` action and instant DOM restoration when toggled off.

7.3 Multi-Platform Abstract Factory (`IPlatformAdapter`)

As stated in `AI lab abstract.pdf`, the architecture is designed to expand to other social video platforms in future releases. All YouTube-specific selectors and SPA lifecycle hooks are isolated inside `YouTubePlatformAdapter`:

```
export interface IPlatformAdapter {
  readonly platformId: 'youtube' | 'tiktok' | 'instagram_reels' | 'x_video';
  extractVideoIdFromUrl(url: string): string | null; // Handles /watch?v=, /shorts/, youtu.be/
  locatePrimaryVideoElement(root: Document): HTMLVideoElement | null;
  observeSpaNavigation(onVideoChange: (newVideoId: string | null) => void): () => void;
  createTranscriptExtractor(): ITranscriptExtractorAdapter;
  createBadgeMountPoint(root: Document): HTMLElement | null;
}
```

Adding support for a new social platform requires implementing only `IPlatformAdapter` and its transcript/badge mount selectors; `ActiveWatchGate`, `SegmentScheduler`, `ObservedIntervalTracker`, `EvidencePackager`, and `IBackendApiAdapter` remain 100% unchanged.

8. Phased Implementation Roadmap, Feasibility Spike & Verification Matrix

To execute the project systematically across the four team members and satisfy all 15 automated test criteria and 13 manual acceptance checks from Section 14 of `extension_scope.md`, development proceeds across eight structured milestones gated by the Phase 2 technical feasibility spike.

Milestone	Deliverable Classes & Modules	Verification & Acceptance Criteria Mapping (Section 14)
M1: Contracts & Core Domain	`TimeInterval`, `VideoSegment`, `EvidencePackage`, all `src/ports/*` interfaces, `MockBackendApiAdapter`	**Tests #4, #5, #6, #7:** Unit-test half-open boundary math `[0, 30)`, partial slice clamping `[10, 30)` & `[30, 48)`, and multi-viewer interval merging in `MockBackendApiAdapter`.
M2: Technical Feasibility Spike	`ChromeVisibleTabScreenshotAdapter`, `ChromeTabAudioCaptureAdapter`, `UnsupportedAudioCaptureAdapter`	Prove visible-tab JPEG capture cadence, MV3 offscreen tab audio loop-back without muting user playback, <100ms timeline drift alignment, and <50ms stop on pause/tab switch. If MV3 audio gesture UX blocks silent activation, bind `UnsupportedAudioCaptureAdapter` as default fallback.
M3: Shell, Identity & Policy Consent	`YouTubePlatformAdapter`, `PolicyConsentManager`, `BrowserPermissionCoordinator`, `UserPreferencesRepository`	**Tests #1, #10, #11:** Verify watch, short-link (`youtu.be`), and Shorts URL parsing; verify one-time policy acceptance per version (`v1`) and zero capture when policy mismatches (`v2`) or contribution is disabled.
M4: Cache & Coverage Lookup	`ChromeStorageResultCacheRepository`, deduplicated lookup queue in `InputCollectionOrchestrator`	**Tests #8, #14:** Verify complete cached results skip collection completely; verify concurrent card lookups are deduplicated and server errors are never cached as classifications.
M5: Active-Watch Gate & Capture	`ActiveWatchGate` (9 specs), `SegmentScheduler`, `ObservedIntervalTracker`, `InputCollectionStateMachine`	**Tests #2, #3, #12, #15:** Verify gate fails on hidden tab, paused/stalled video, or background window; verify immediate stop and bounded partial flush on navigation; verify zero raw media in `chrome.storage.local`.
M6: Transcript Extraction	`YouTubeDomTranscriptAdapter`, `MockTranscriptExtractorAdapter`, SHA-256 content hasher	**Test #9:** Verify strict distinction between `full`, `partial`, and `unavailable` transcripts and single-upload deduplication via `content_hash`.
M7: Badges & Filtering UI	`BadgePresenter`, `MarkOnlyFilterStrategy`, `ThresholdHideFilterStrategy`	**Test #13:** Verify 7 badge states; verify filtering defaults to `mark_only` and never hides partial, `UNKNOWN`, pending, or failed videos; verify session reveal and restore.
M8: Hardening & Full QA	`HttpBackendApiAdapter` (swapping out Mock Adapter once backend schemas are locked), E2E suite	**Tests #14, #15 + 13 Manual Checks:** Rate-limit `Retry-After` handling, payload size enforcement, manifest permission audit (`storage`, `activeTab`, host permissions only), and full manual QA checklist sign-off.

8.1 Dependency Injection & Adapter Wiring Example (Mock vs. Production Mode)

By wiring the extension entrypoint through a clean `ExtensionContainer` factory, developers can run the entire extension on YouTube today using Mock Adapters for unresolved backend/capture parameters, and switch individual adapters to production via environment flags as each Section 16 decision is finalized:

```
export function createInputCollectionOrchestrator(env: 'development_mock' | 'production'):
InputCollectionOrchestrator {
  const platformAdapter: IPlatformAdapter = new YouTubePlatformAdapter();
  const segmentationStrategy: ISegmentationStrategy = new ConfigurableFixedSegmentationStrategy(30);

  const backendAdapter: IBackendApiAdapter = env === 'development_mock'
    ? new MockBackendApiAdapter({ defaultSegmentDurationSeconds: 30, simulateNetworkLatencyMs: 100 })
    : new HttpBackendApiAdapter({ baseUrl: 'https://api.aidetect-crowdsourced.org', uploadFormat: 'multipart' });

  const screenshotAdapter: IScreenshotCaptureAdapter = env === 'development_mock'
    ? new MockScreenshotCaptureAdapter({ intervalSeconds: 5, maxFramesPerSegment: 6 })
    : new ChromeVisibleTabScreenshotAdapter({ intervalSeconds: 5, maxFramesPerSegment: 6, maxWidthPx: 1280,
jpegQuality: 0.75 });

  const audioAdapter: IAudioCaptureAdapter = env === 'development_mock'
    ? new MockAudioCaptureAdapter()
    : ChromeTabAudioCaptureAdapter.isSupportedInCurrentContext()
      ? new ChromeTabAudioCaptureAdapter({ chunkDurationSeconds: 5, bitrateBps: 32000 })
      : new UnsupportedAudioCaptureAdapter('gesture_required');

  const transcriptAdapter: ITranscriptExtractorAdapter = platformAdapter.createTranscriptExtractor();

  return new InputCollectionOrchestrator(
```

```
    platformAdapter,
    segmentationStrategy,
    backendAdapter,
    screenshotAdapter,
    audioAdapter,
    transcriptAdapter,
    new StoragePolicyConsentAdapter('v1'),
    new ChromeStorageResultCacheRepository({ maxEntries: 500, ttlSeconds: 86400 })
  );
}
```

8.2 Automated Unit & Integration Test Harness Architecture

Because every external browser API and backend dependency is injected through constructor interfaces, all 15 automated test suites mandated in Section 14 run deterministically in Node.js (Vitest/Jest) in under 2 seconds without launching a full browser instance:

- **Deterministic Clock & Playback Simulation:** `FakeVideoPlaybackClock` advances simulated `video.currentTime` in 250ms ticks, allowing tests to assert exact half-open interval slicing across segment boundaries (`[0, 30)`, `[30, 60)`), user seeks, and buffer stalls.
- **Multi-Viewer Convergence Simulation:** Two independent `InputCollectionOrchestrator` instances connected to a shared `MockBackendApiAdapter` instance verify that Viewer A (watching `10s–48s`) and Viewer B (watching `0s–10s` and `48s–60s`) atomically converge Segments 0 and 1 to `complete` while leaving Segments 2 and 3 in `missing` state.
- **Privacy & Storage InvariantSpy:** `SpyChromeStorageArea` intercepts every write to `chrome.storage.local` across all test suites and throws an immediate assertion error if any payload contains binary buffers, Base64 image/audio strings, or raw transcript text.